Are you stressed? Bayesian Regime Detection in Crypto Markets

Learning Which Data Matters for Scalable HMM Inference

Diego Luca Gonzalez Gauss, Enrico Yao-Bate, Haozhe Yang

STAT 221 Spring 2026 Final Project

May 5, 2026

The high-level question

Can we do scalable Bayesian regime inference for crypto returns?

Why this is interesting

Crypto markets switch between calm, stress, and chaos regimes. If we can infer which regime we are in, fast and with calibrated uncertainty, we get risk control and drawdown avoidance.

Why this is hard

The natural model for regime-switching is a Hidden Markov Model (HMM). But HMM posteriors are expensive to compute, and the standard tools for scaling inference introduce new problems of their own.

Why an HMM for crypto returns?

Crypto assets go through extended periods of low volatility where daily returns are small and centered near zero. Then, abruptly, daily moves of 10-20% become common, correlations spike, and drawdowns cascade across assets. These shifts are discrete, not gradual. An HMM captures this:

- Latent state $z_t \in \{1, \ldots, K\}$: market regime.
- Return distribution depends on the regime:

$$y_t \mid z_t = k \sim \mathcal{N}(\mu_k, \Sigma_k).$$

- Regimes tend to persist, i.e. if we are in calm today, we are likely in calm tomorrow:

$$z_t \mid z_{t-1} \sim \text{Categorical}(A_{z_{t-1},:}).$$

Example states

0: calm / low volatility.
1: stressed / increasing volatility.
2: chaos / extreme volatility

Why MCMC?

We want the full posterior $p(z_t \mid y_{1:T}, \boldsymbol{\theta})$ over model parameters and latent states. But it is complex because it is high-dimensional (transition matrix, emission parameters for each state) and involves discrete latent variables.

- EM gives a point estimate but no uncertainty quantification.
- Variational methods require choosing an approximating family, which can miss important posterior structure.
- MCMC targets the exact posterior.

The problem

MCMC requires the full posterior gradient at every step. For an HMM, this means an $O(TK^2)$ forward-backward pass over the entire series. For long series, this is prohibitive.

Why stochastic-gradient MCMC?

The idea is that because the HMM log-posterior decomposes as a sum over time, instead of computing the full gradient, we can approximate it by sampling M block indices $I_1, \ldots, I_M$ from a distribution p over the B blocks, and reweighting by their sampling probabilities:

$$\widehat{G}(\boldsymbol{\theta}) = \nabla \log p(\boldsymbol{\theta}) + \frac{1}{M} \sum_{m=1}^{M} \frac{g_{I_m}(\boldsymbol{\theta})}{p_{I_m}}, \qquad g_b(\boldsymbol{\theta}) = \nabla \ell_b(\boldsymbol{\theta}).$$

SGLD update (Welling & Teh, 2011):

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \frac{\eta_t}{2} \widehat{G}(\boldsymbol{\theta}_t) + \xi_t, \qquad \xi_t \sim \mathcal{N}(0, \eta_t I).$$

The lower-level question

How should we set the block sampling distribution p?

A first-pass answer is uniform sampling $p_b = 1/B$ since it is unbiased and simple. However, it ignores the fact that not all blocks are equally informative.

In crypto, most blocks capture sideways price action, and only a few contain crashes, liquidation cascades, or regime transitions. These rare blocks carry most of the gradient signal for stress-regime parameters, meaning that if we consistently under-sample them, we may lose the ability to accurately detect when the market is entering or exiting a stress regime.

The statistical computing question

How should we set the block sampling distribution p in SG-MCMC for HMMs to ensure rare but informative regimes are adequately represented in the gradient estimate?

An existing answer: TASS (Ou, Young, Sen, Dunson, 2024)

Their answer

Sample each block in proportion to how much gradient signal it carries, so blocks that matter more for the posterior get sampled more often.

Formally, we want to choose sampling weights a_n that minimize the variance of our stochastic gradient estimator $\mathbb{E}\|\widehat{\nabla} - \nabla\|^2$. If we knew the latent states z_t, we could compute each block's gradient contribution

$$g_n(\boldsymbol{\theta}) = \sum_{t \in C_n} \nabla_{\boldsymbol{\theta}} \left[\log p(y_t \mid z_t) + \log p(z_t \mid z_{t-1})\right].$$

and just as we set proposal weights proportional to the integrand in importance sampling, the variance-optimal thing to do here is to set block weights $a_n^\star$ proportional to gradient norms.

Problem: these weights require knowing the latent states z_t, which are exactly what we are trying to infer, and the current parameter $\boldsymbol{\theta}$, which changes at every MCMC step.

How TASS approximates optimal weights in practice

1. **Cluster observations:** run K-means on $y_1, \ldots, y_T$ to get proxy state labels $\hat{z}_t$.
2. **Compute approximate block gradients:** evaluate the block gradient formula with $\hat{z}_t$ in place of z_t and $\boldsymbol{\theta}$ fixed at $\boldsymbol{\theta}_{\mathrm{MAP}}$:

$$\tilde{g}_n = \sum_{t \in C_n} \nabla_{\boldsymbol{\theta}} \left[\log p(y_t \mid \hat{z}_t) + \log p(\hat{z}_t \mid \hat{z}_{t-1}) \right] \Big|_{\boldsymbol{\theta} = \boldsymbol{\theta}_{\mathrm{MAP}}}.$$

3. **Set weights:** $\tilde{a}_n = \dfrac{\|\tilde{g}_n\|}{\sum_{n'=1}^{N} \|\tilde{g}_{n'}\|}$ so blocks with many rare-state observations get large $\tilde{a}_n$.
4. **Sample:** draw blocks with probabilities $p_n^{\mathrm{TASS}} = (1 - \lambda)\, \tilde{a}_n + \dfrac{\lambda}{N}$, where the λ/N is to ensure every block still has a chance to be sampled.

Limitation

TASS weights are static. They only use emission values (through K-means), and cannot adapt if the importance of blocks shifts as the chain explores the posterior.

Our answer: learn which blocks matter during sampling

High-level idea

Replace the static weights with a model that predicts each block's gradient importance from observable features, and updates those predictions during sampling using gradient feedback.

For each block C_n, we compute a feature vector $w_n \in \mathbb{R}^p$ from the observations in that block:

- mean absolute return, mean squared return,
- rolling volatility (within-block standard deviation),
- cross-sectional dispersion (spread across similar assets),
- tail event counts (number of returns exceeding a threshold),
- downside pressure (fraction of negative returns).

These are observable proxies for regime activity since stress blocks tend to have high volatility, large returns, and high dispersion.

The online weight model

Recall from TASS that the optimal weight for block n is $a_n^\star \propto \|g_n(\boldsymbol{\theta})\|$. We want to predict this from the block features w_n, and update the prediction as $\boldsymbol{\theta}$ changes during sampling.

We opt to do this by turning it into a regression problem. Gradient norms are positive and vary over orders of magnitude, so we model the log-squared norm as linear in features:

$$\log \|g_n(\boldsymbol{\theta}_t)\|^2 \;\approx\; \widehat{\boldsymbol{\beta}}_t^\top w_n,$$

where $\widehat{\boldsymbol{\beta}}_t \in \mathbb{R}^p$ is a coefficient vector, updated at each MCMC step t, that captures how each feature relates to gradient importance. This gives natural importance weight estimates

$$\widehat{a}_{n,t} = \exp\!\left(\tfrac{1}{2}\,\widehat{\boldsymbol{\beta}}_t^\top w_n\right) \;\approx\; \|g_n(\boldsymbol{\theta}_t)\|\,.$$

As in TASS, we convert weights to sampling probabilities with a uniform exploration floor:

$$p_{n,t} = (1-\lambda)\,\frac{\widehat{a}_{n,t}}{\sum_{n'=1}^{N} \widehat{a}_{n',t}} + \frac{\lambda}{N}.$$

Fitting $\widehat{\boldsymbol{\beta}}_t$ by discounted ridge regression

At each SGLD step t, after sampling block C_{n_s} and computing its gradient, we observe what we call the **feedback response** for our regression:

$$f_s = \log\left(\|g_{n_s}(\boldsymbol{\theta}_s)\|^2 + \epsilon\right),$$

where $\epsilon > 0$ prevents $\log 0$. We now have a training pair (w_{n_s}, f_s) and can update $\widehat{\boldsymbol{\beta}}_t$.

We need a regression method that (a) adapts to non-stationarity, since block importance shifts as $\boldsymbol{\theta}$ moves, and (b) has a cheap closed-form update, since it runs inside every SGLD step. Discounted ridge regression satisfies both, so we do

$$\widehat{\boldsymbol{\beta}}_t = \arg\min_{\boldsymbol{\beta}} \sum_{s<t} \rho^{t-s}\left(f_s - \boldsymbol{\beta}^\top w_{n_s}\right)^2 + \tau \|\boldsymbol{\beta}\|^2,$$

where the **discount** $\rho \in (0,1)$ down-weights old observations exponentially, so the model tracks shifting importance rather than averaging over the entire chain history, while our **ridge penalty** $\tau > 0$ regularizes early in the chain when few blocks have been sampled.

This has a closed-form recursive update ($O(p^2)$ per step), so it is cheap enough to run inside every SGLD iteration

When does online beat static?

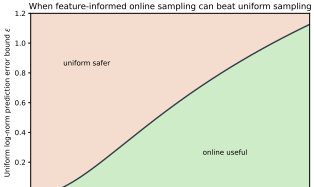
Write $a_n = \|g_n(\boldsymbol{\theta})\|$ for the true gradient norm of block n. The stochastic gradient variance under uniform sampling is inflated by a factor of

$$1 + \mathrm{CV}(a)^2, \qquad \text{where } \mathrm{CV}(a) = \frac{\mathrm{sd}(a_1, \ldots, a_N)}{\mathrm{mean}(a_1, \ldots, a_N)}.$$

When gradient norms are heterogeneous across blocks (high CV), uniform sampling wastes most of its budget on low-signal blocks.

Online beats uniform whenever its prediction error ε (worst-case log-ratio between predicted and true weights) satisfies

$$\frac{e^{2\varepsilon}}{1-\lambda} < 1 + \mathrm{CV}(a)^2.$$



Scale down to an actual crypto task

Forward-looking target

At the end of each 10-day block, predict whether the next 10-day equal-weight crypto basket return is a stress event:

$$y_b = \mathbf{1}\{R_{b+1}^{\mathrm{mkt}} \leq -7\%\}.$$

Features

Current-block return, volatility, drawdown, downside pressure, cross-asset dispersion, tail counts, and asset-specific vol/downside summaries.

This is a real crypto question

Should we be risk-on or risk-off for the next two weeks?

Why this task is covered by the math

Fit a logistic stress model:

$$h_\beta(w_b) = \sigma(\beta^\top w_b), \qquad \ell_b(\beta) = \text{cross-entropy}(y_b, h_\beta(w_b)).$$

The block gradient is

$$g_b(\beta) = \nabla \ell_b(\beta) = \{h_\beta(w_b) - y_b\} w_b.$$

Same useful-regime condition

If a few blocks are high-loss/high-gradient, and cheap crypto features predict them, online weighting should improve stochastic training.

Why static can get stale

The stress signature changes: volatility, downside skew, dispersion, liquidation-like tails. Online can track; frozen weights cannot.

Actual crypto task results

Method	Log loss	Brier	AUC	Top-risk stress rate	Strategy return	Max drawdown
Online	0.524	0.172	0.625	37.5%	-33.3%	-49.5%
Static TASS	0.531	0.175	0.623	25.0%	-25.0%	-48.3%
Uniform	0.533	0.176	0.620	25.0%	-41.5%	-55.1%
Buy-and-hold	–	–	–	–	-49.7%	-65.1%

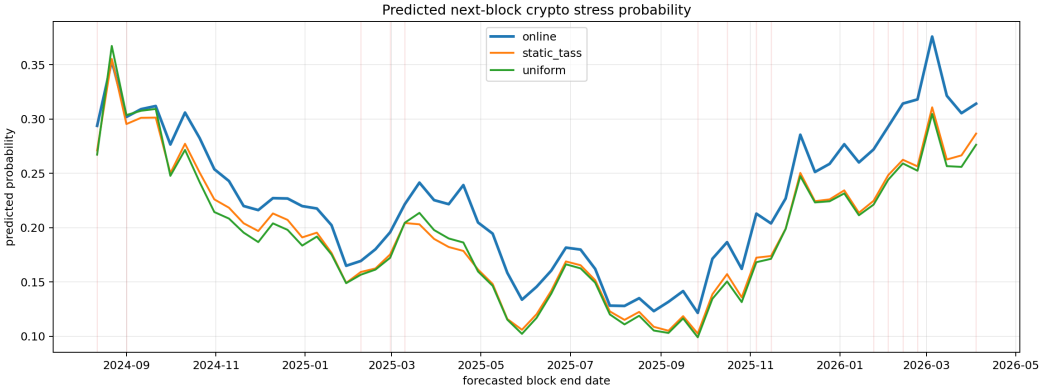
Readout

Online has the best log loss, Brier score, AUC, and top-risk stress concentration.

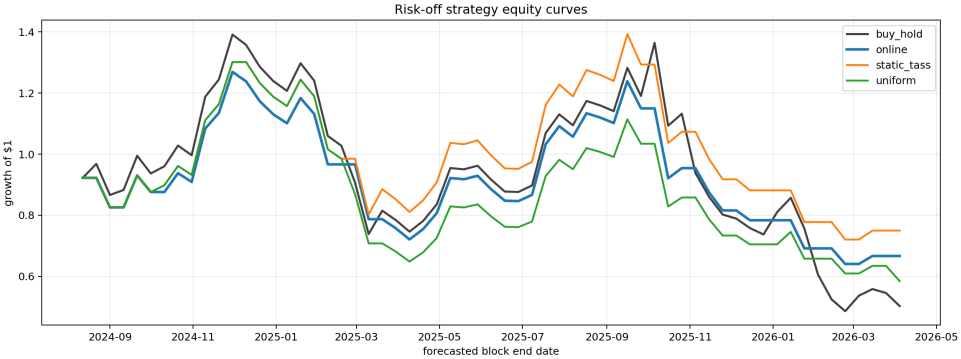
Trading caveat

Static TASS wins this simple total-return threshold rule. Online is the cleanest risk forecaster; trading P&L is a noisy downstream functional.

Online risk scores over time



Money-flavored diagnostic



Interpretation

The risk-off rule is intentionally simple: hold crypto unless predicted stress is top-quartile, then hold cash.

Then we tried to scale to full HMM inference

The hope

Use the same targeted weighting idea inside SGLD for Bayesian HMM posterior sampling.

The reality

Once the target is a full HMM posterior, gradient variance is no longer the only bottleneck.

- Label switching and multimodality.
- Weakly identified third regime.
- Transition/covariance coupling.
- Finite-step SGLD bias and numerical stability constraints.

What happened empirically

Original $K = 3$ Student-t crypto model

Collapsed into state 0 over 99% of the time. Expensive and not interpretable.

$K = 2$ rolling-standardized Gaussian model

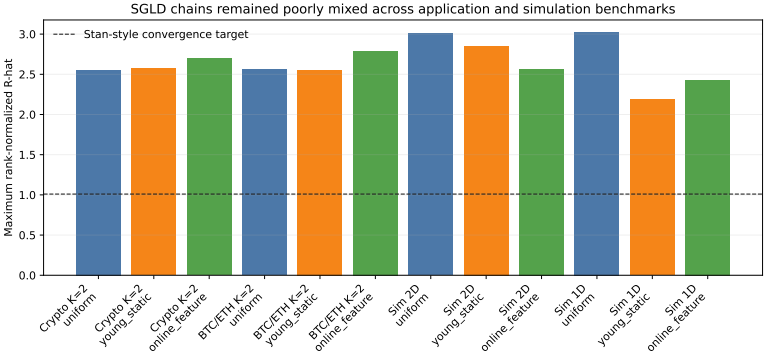
Much healthier occupancy: roughly 60/40 calm vs stress states.

But

SGLD chains still mixed poorly. Stan/NUTS mixed well for $K = 2$, and failed for $K = 3$.

The useful visual result

The painful diagnostic result



Interpretation

Good-looking state probabilities are not enough. Cross-chain diagnostics say the SGMCMC posterior samples are not trustworthy yet.

Why this does not kill the online idea

- Online weighting is a gradient-estimator variance reduction tool.
- Our failures were dominated by HMM posterior geometry:
  - label switching,
  - weakly identified third state,
  - transition/covariance coupling,
  - finite-step SGLD bias.
- If posterior geometry is the bottleneck, better block weights cannot magically fix the chain.

Cleaner claim

Online weighting deserves a benchmark where gradient-estimator variance is the bottleneck and feature-gradient relationships evolve over time.

Takeaways

1. TASS is principled: sample subsequences in proportion to parameter-specific gradient importance.
2. Online weighting helps when importance is heterogeneous, feature-predictable, and nonstationary.
3. The actual crypto stress task matches that regime; online is the best risk forecaster.
4. $K = 2$ rolling-standardized Gaussian HMM is the defensible Bayesian regime model.
5. The SGMCMC chains did not mix well enough for posterior claims.

The mooolah is not in forcing SGLD to work everywhere.
It is in using adaptivity where the math says it should pay.

References

Ou, R., Young, A. L., Sen, D., and Dunson, D. B. (2024). *Targeted Stochastic Gradient MCMC for HMMs with Rare Latent States*. Bayesian Analysis.

Welling, M. and Teh, Y. W. (2011). *Bayesian Learning via Stochastic Gradient Langevin Dynamics*. ICML.

Vehtari, A., Gelman, A., Simpson, D., Carpenter, B., and Bürkner, P.-C. (2021). *Rank-normalization, folding, and localization: An improved R-hat for assessing convergence of MCMC*. Bayesian Analysis.